

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.....	5
1.1 Анализ существующих систем для тестирования сотрудников.....	5
1.2 Классификация типов тестов и вопросов.....	11
1.3. Методика оценки результатов тестирования.....	15
2. ПРАКТИЧЕСКАЯ ЧАСТЬ.....	19
2.1 Проектирование ИС.....	19
2.1.1 Описание функционала информационной системы.....	19
2.1.2 Концептуальное проектирование информационной системы.....	21
2.1.3 Проектирование структуры базы данных.....	22
2.1.4 Разработка структуры с учетом модели данных.....	23
2.2 Реализация информационной системы.....	27
2.2.1 Реализация базы данных.....	27
2.2.2 Создание информационной системы.....	30
2.3 UML-Диаграмма.....	41
ЗАКЛЮЧЕНИЕ.....	43
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	44
ПРИЛОЖЕНИЕ 1.....	47
ПРИЛОЖЕНИЕ 2.....	48
ПРИЛОЖЕНИЕ 3.....	49
ПРИЛОЖЕНИЕ 4.....	51
ПРИЛОЖЕНИЕ 5.....	52
ПРИЛОЖЕНИЕ 6.....	54

ВВЕДЕНИЕ

В условиях современной экономики ключевым фактором конкурентоспособности любого предприятия становится уровень профессиональной компетентности его персонала. Регулярная оценка знаний сотрудников необходима для выявления пробелов в подготовке, планирования обучения и аттестации. Однако традиционные методы проведения тестирования (бумажные анкеты, опросы в режиме реального времени, рассылка файлов по электронной почте) обладают рядом недостатков: высокие временные затраты на проверку результатов, трудоемкость обработки статистики, а также риск человеческой ошибки при оценке.

На текущий момент на предприятии "ООО «Полимер»" отсутствует единая централизованная система контроля знаний, что приводит к необъективности результатов и снижению мотивации персонала к самообучению. В связи с этим задача автоматизации процесса сбора и анализа ответов сотрудников становится актуальной.

Объект исследования – автоматизация процесса проведения внутренних аттестационных тестов для выявления уровня знаний работников предприятия.

Предмет исследования – информационная система (ИС), выявляющая уровень знаний работников для предприятия Полимер.

Цель – создание корпоративного веб-сайта для автоматизации процесса входного тестирования работников предприятия.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Изучить и описать предметную область.
2. Сформулировать функциональные и нефункциональные требования к информационной системе.

3. Создать структуру базы данных.
4. Разработать информационную систему.

Результатом работы станет действующий сайт, который позволит руководству предприятия оперативно выявлять сотрудников с недостаточным уровнем знаний, формировать группы для до обучения, а также снизить нагрузку на HR-отдел за счет полной автоматизации проверки тестов.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Анализ существующих систем для тестирования сотрудников

В современных условиях цифровой трансформации кадровых процессов автоматизированные системы тестирования сотрудников становятся неотъемлемым инструментом HR-служб. Основное назначение таких систем – объективная и оперативная оценка профессиональных знаний, навыков и компетенцией персонала, минимизация субъективного фактора при проведении аттестации, а также сокращение временных и трудовых затрат на подготовку, проведение и анализ результатов тестирований.

Существующие программные решения можно классифицировать по нескольким критериям:

1. По способу развертывания: облачные сервисы (SaaS) и локальные (on-premise) системы.
2. По специализации: узкоспециализированные (только тестирование) и комплексные HRM-системы (включающие обучение, оценку, управление талантами).
3. По модели распространения: проприетарное (коммерческое) и свободное программное обеспечение (open-source).
4. По степени интеграции: автономные системы и встраиваемые модули (например, в корпоративные порталы или LMS).

Для анализа были выбраны наиболее репрезентативные системы, получившие распространение на российском и международном рынках:

1. Moodle (Рисунок 1).

Тип: свободная система управления обучением (LMS).

Возможности для тестирования: наличие мощного модуля «Тест», поддерживающего более 15 типов вопросов (в том числе эссе,

вычисляемые, на соответствие, случайные), гибкую настройку весовых коэффициентов, таймеры, адаптивный режим (сложность меняется от ответов), безопасный режим с IP-контролем [29].

Достоинства: огромное сообщество, тысячи плагинов, бесплатность, локализация на русский язык [29].

Недостатки: избыточность функционала для задач простого тестирования, сложность администрирования для не-IT специалиста, высокий порог входа для настройки отдельного теста [29].

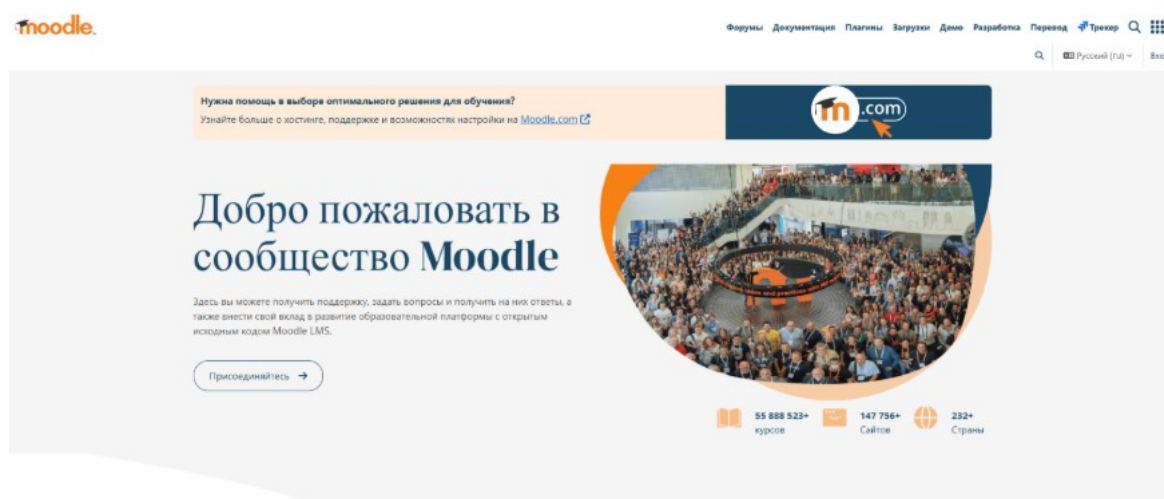


Рисунок 1. Скриншот стартовой страницы Moodle

2. iSpring Online (Рисунок 2).

Тип: коммерческий конструктор курсов и облачная платформа тестирования.

Возможности: глубокая интеграция с PowerPoint, создание тестов с ветвлением сценариев, видео-вопросы, возможность установки проходного балла, детальная статистика по каждому вопросу. Доступна продакшн-версия для корпоративных порталов [28].

Достоинства: удобный интерфейс, быстрая конвертация существующих презентаций в тесты, качественная русскоязычная поддержка [28].

Недостатки: высокая стоимость лицензий (от 50 000 руб./год), привязка к экосистеме iSpring, невозможность глубокой кастомизации внешнего вида под бренд компании[28].

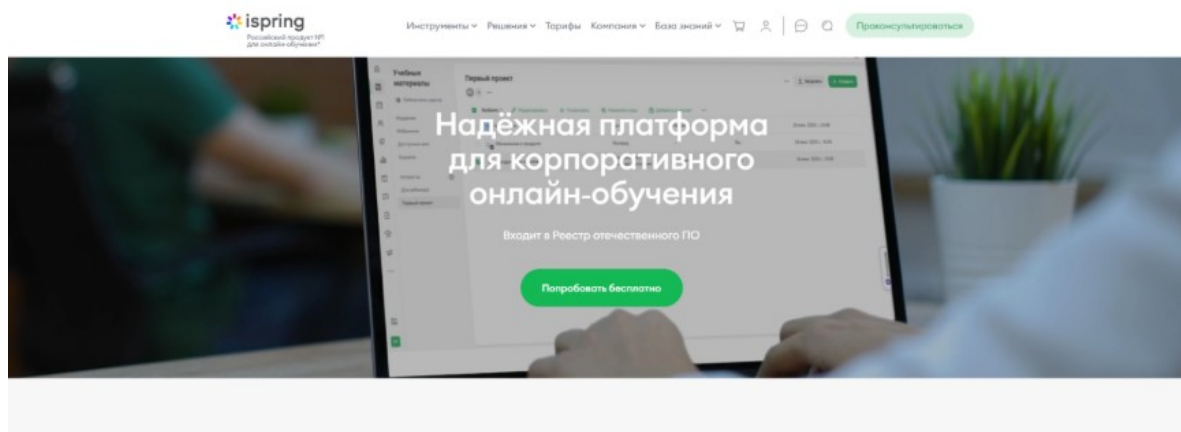


Рисунок 2. Скриншот стартовой страницы iSpring

3. Google Forms (Рисунок 3).

Тип: бесплатный онлайн-конструктор форм (условно – облачный).

Возможности: создание тестов с автоматической проверкой закрытых вопросов, сбор ответов в Google Sheets, базовая настройка весов баллов [25].

Достоинства: нулевая стоимость, интуитивно понятный интерфейс, коллективный доступ к результатам [25].

Недостатки: отсутствие ролевой модели (администратор/тестируемый), нет таймера на отдельный тест, нельзя ограничить количество попыток или время прохождения, слабая защита от списывания, отсутствие паспорта сотрудника (ФИО может быть указано любое), нет экспорта в PDF/Excel с единой фирменной отчетностью [25].

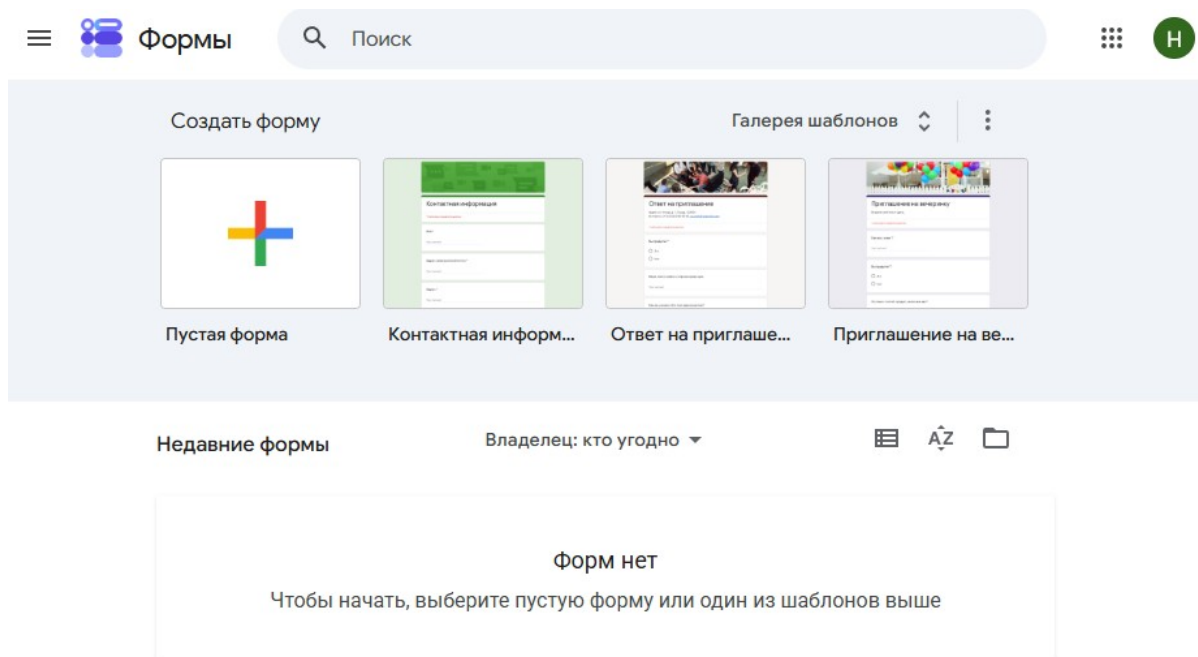


Рисунок 3. Скриншот стартовой страницы Google Forms

4. Testograf (Рисунок 4).

Тип: коммерческий облачный сервис для тестирования (русская разработка).

Возможности: конструктор тестов с 12 типами вопросов, настройка логики переходов, сертификация результатов, API для интеграции с корпоративными системами [13].

Достоинства: специализированное решение именно для тестирования, хорошая документация [13].

Недостатки: платный план для более чем 50 тестирований в месяц (от 2 000 руб./мес.), отсутствие open-source версии, невозможность локального развертывания на сервере компании (что критично для организаций с ограничениями по передаче персональных данных) [13].

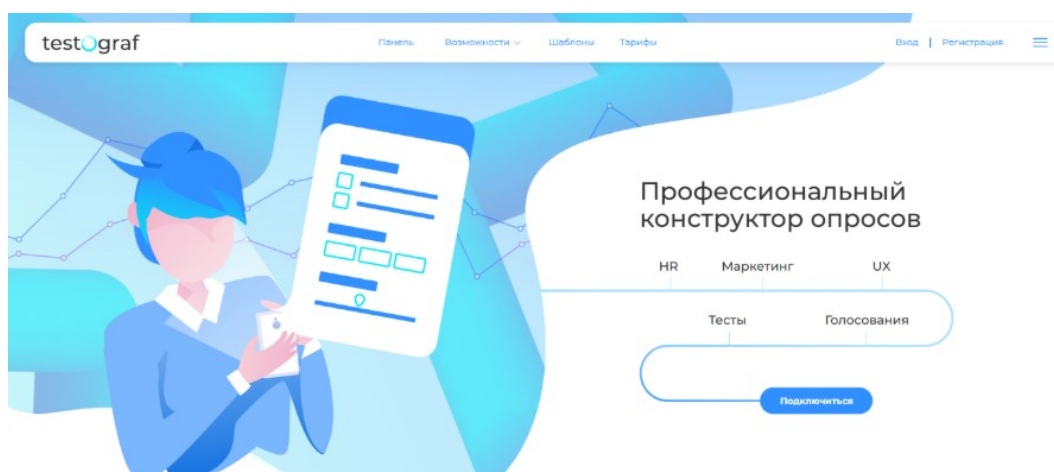


Рисунок 4. Скриншот стартовой страницы Testograf

На основе изучения вышеперечисленных систем составлена сравнительная таблица 1 по критериям, значимым для коммерческой организации [13, 25, 28, 29].

Таблица 1 – Сравнение систем тестирования

Критерий	Moodle	iSpring	Google Forms	Testograf
Бесплатность / низкая стоимость	Да	Нет	Для малых	Нет
Ролевая модель (админ → HR → сотрудник)	Да	Да	Нет	Да
Таймер и ограничение попыток	Да	Да	Нет	Да
Индивидуальный отчет по сотруднику в PDF	Да	Да	Нужен макрос	Да
Защита от пересдачи / подмены пользователя	Частично	Да	Нет	Да
Возможность локальной установки	Да	Нет	Нет	Нет
Адаптивный дизайн (мобильные устройства)	Да	Да	Да	Да
Поддержка русского языка (интерфейс)	Да	Да	Да	Да

Проведенный анализ позволяет сформулировать следующие выводы:

1. Существующие бесплатные системы (Google Forms, часть open-source) не удовлетворяют требованиям полноценной аттестации сотрудников из-за отсутствия ролевой модели, механизмов защиты результатов и кастомизируемой отчетности [25].

2. Коммерческие системы (iSpring, Testograf) обладают нужным функционалом, но имеют высокую стоимость подписки (особенно при масштабировании) и, что критично для ряда организаций, не позволяют размещать данные на внутреннем сервере компании в правовом поле 152-ФЗ (о персональных данных) [1, 13, 28].

3. Универсальные LMS (Moodle) избыточны: для задач тестирования не требуется модуль форума, глоссарий, wiki и прочие средства обучения. Администрирование Moodle силами HR-отдела без программиста затруднительно [29].

Таким образом, целесообразна разработка собственного легковесного веб-сайта для тестирования сотрудников, который:

- реализует необходимый минимум функций (авторизация, роли, создание тестов, формирование отчетов);
- разворачивается на внутреннем сервере компании (обеспечивая безопасность персональных данных);
- не требует лицензионных отчислений;
- обладает понятным интерфейсом, адаптированным для неопытных пользователей;
- позволяет в перспективе дорабатывать функционал силами штатных разработчиков.

Данное решение закрывает пробел между дорогими коробочными продуктами и слишком примитивными бесплатными инструментами.

1.2 Классификация типов тестов и вопросов

Для разработки корректной и функциональной информационной системы тестирования сотрудников необходимо чёткое понимание форматов заданий, которые будут использоваться при аттестации. От выбора типов вопросов зависят:

- структура базы данных (способ хранения вопроса, вариантов ответа, правильных ответов);
- алгоритм подсчёта результатов (проверка точного совпадения, частичного совпадения, весовых коэффициентов);
- пользовательский интерфейс (радиокнопки, чекбоксы, текстовое поле, выпадающий список).

Кроме того, корректная классификация позволяет HR-специалистам создавать тесты, соответствующие целям оценки: проверка знаний, диагностика мышления или оценка поведения.

В психометрике и педагогических измерениях принято выделять следующие группы тестов [2, 8]:

1. По степени стандартизации:
 - стандартизированные (все сотрудники получают одинаковые вопросы в одинаковом порядке) – обеспечивают сопоставимость результатов;
 - нестандартизированные (порядок вопросов или их набор может варьироваться) – снижают вероятность списывания, но усложняют анализ.
2. По способу предъявления:
 - последовательные (вопрос за вопросом без возврата) – наиболее просты в реализации и привычны для тестируемых;

– навигационные (можно переключаться между вопросами, возвращаться к пропущенным) – удобны, но требуют сохранения всех ответов на стороне сервера;

– адаптивные (следующий вопрос зависит от правильности предыдущего) – сложны в реализации, но позволяют быстрее оценить уровень сотрудника.

3. По временному режиму:

– с ограничением общего времени (тест рассчитан на 30 минут) классический вариант, поддерживается большинством систем;

– с ограничением времени на отдельный вопрос (например, 30 секунд на задание) – требует дополнительных проверок на клиенте и сервере;

– без ограничения времени – используется для квалификационных работ или опросов.

При проектировании веб-сайта для тестирования сотрудников следует опираться на наиболее распространённые и востребованные в кадровой практике типы вопросов. Ниже представлены таблицы 2 – 4 с указанием особенностей алгоритмической обработки.

Таблица 2 – Закрытые вопросы

Подтип	Описание	Выбор ответа	Алгоритм проверки
Одиночный выбор	Из нескольких вариантов верен только один	один	сравнение ID выбранного варианта с эталоном
Множественный выбор	Может быть, несколько верных ответов	один или несколько	сравнение множества выбранных ID с эталонным множеством (полное совпадение) или частичное совпадение с весами
Выпадающий список	Варианты скрыты до клика	один	аналогично одиночному выбору
Верно / Неверно	Утверждение, которое нужно оценить	да/нет	сравнение с булевым эталоном

Таблица 3 – Открытые вопросы

Подтип	Описание	Ввод ответа	Алгоритм проверки
Краткий ответ	Слово, число, код	текстовая строка	нормализация (приведение к нижнему регистру, удаление лишних пробелов) и точное или нечёткое совпадение
Числовой ответ	Результат вычисления	число	сравнение с эталоном с допустимой погрешностью
Эссе	Развёрнутое рассуждение	многострочный текст	не автоматизирована (оценивается HR-менеджером вручную)

Таблица 4 – Вопросы на соответствие и упорядочивание

Тип	Описание	Интерфейс	Алгоритм проверки
На соответствие	Сопоставить элементы из двух списков	Drag-and-drop или два выпадающих списка	попарное сравнение (счётчик правильных пар)
На установление последовательности	Расставить шаги процесса в правильном порядке	Сортируемый список	сравнение порядка индексов с эталонной последовательностью (алгоритм Левештейна для частичного совпадения)

Для обоснования функциональных требований к разрабатываемой информационной системе необходимо учитывать различные сценарии использования (Классификация тестов по цели проведения) [11]. Анализ сценариев позволяет выявить полный спектр действий пользователей, их роли, цели и возможные варианты взаимодействия с системой в зависимости от контекста. Такой подход способствует формированию полных, непротиворечивых и проверяемых требований, покрывающих реальные бизнес-процессы, а также помогает предотвратить пропуск критически важного функционала, снизить риски ошибок на этапе

разработки и обеспечить удобство использования системы в разнообразных условиях эксплуатации [16]. (Таблица 5)

Таблица 5 – Классификация тестов по цели проведения

Разновидность теста	Цель	Особенности вопросов	Необходимый функционал ИС
Входной / квалификационный	Проверка знаний при приёме или переводе	закрытые + краткий ответ	защита от подсказок, таймер
Периодическая аттестация	Оценка сотрудников за период	смешанные типы, кейсы	генерация отчётов для руководителя, история попыток
Экзамен / сертификация	Допуск к работе	преимущественно закрытые	строгий таймер, запрет возврата к вопросам, фиксация IP
Самооценка / тренировка	Подготовка сотрудника	открытые, с подсказками	режим «показать правильный ответ», неограниченные попытки

Для наглядного обоснования выбора при реализации собственной системы приведём сравнительную таблицу 6 поддержки различных типов вопросов в системах [13, 25, 28, 29], рассмотренных в таблице 1.

Таблица 6 – Матрица поддержки типов вопросов в типовых системах

Тип вопроса	Moodle	iSpring	Google Forms	Testograf
Одиночный выбор	Есть	Есть	Есть	Есть
Множественный выбор	Есть	Есть	Есть	Есть
Краткий ответ	Есть	Есть	Только точное совпадение	Есть
Числовой с погрешностью	Есть	Есть	Нет	Есть
Эссе	Есть	Есть	Есть	Есть
Соответствие	Есть	Есть	Нет	Есть
Упорядочивание	Есть	Есть	Нет	Есть

При создании тестов важно не только правильно выбрать тип вопроса, но и настроить его «вес» в общей сумме баллов [2]. В разрабатываемой ИС предлагается использовать следующую систему:

- базовая сложность (коэффициент 1.0) – вопрос на воспроизведение факта (одиночный выбор или верно/неверно);
- средняя сложность (коэффициент 1.5) – множественный выбор (нужно учесть все верные варианты) или краткий ответ с вводом термина;
- повышенная сложность (коэффициент 2.0) – числовой ответ с вычислениями или вопрос на соответствие.

1.3. Методика оценки результатов тестирования

Методика оценки результатов тестирования является ключевым компонентом любой аттестационной системы, поскольку именно она определяет, каким образом сырые ответы сотрудника преобразуются в содержательный вывод о его уровне знаний или компетенцией [2, 8]. Непродуманная или необъективная методика может свести на нет все преимущества автоматизированного тестирования, привести к демотивации персонала и юридическим спорам при принятии кадровых решений.

Для разрабатываемой ИС необходимо реализовать методику, отвечающую следующим требованиям:

- прозрачность – сотрудник должен понимать, за что и сколько баллов он получил;
- воспроизводимость – одинаковые ответы должны давать одинаковый результат независимо от времени прохождения;
- гибкость – администратор должен иметь возможность настраивать параметры оценки под разные типы тестов;

– интерпретируемость – числовой балл должен однозначно переводиться в категорию (например, «сдал», «не сдал», «отлично»).

В таблице 7 приведен процесс оценки результатов тестирования в автоматизированной системе [2, 5, 8]:

Таблица 7 – Этапы процесса оценки результатов

Этап	Описание	Отвечающий модуль в ИС
Сбор ответов	Фиксация выбранных / введенных пользователем вариантов	Клиентская часть → Отправка на сервер
Нормализация ответов	Приведение к стандартному виду (удаление лишних пробелов, приведение регистра, приведение числовых форматов)	Серверный модуль предобработки
Сопоставление с эталонами	Сравнение ответа пользователя с правильным(и) ответом(ами) в зависимости от типа вопроса	Алгоритм проверки
Вычисление первичных баллов	Начисление весовых коэффициентов, суммирование, расчёт процентов	Модуль подсчёта
Интерпретация	Перевод процентного результата в итоговую оценку / категорию	Модуль интерпретации
Сохранение и генерация отчёта	Запись результата в БД, формирование документа для сотрудника / руководителя	Модуль отчётности

Для перевода первичных баллов (процентов) в результат используется шкала оценивания [2]. (Таблица 8) Предлагается гибкая система, позволяющая администратору настраивать шкалу для каждого теста индивидуально.

Таблица 8 – Шкала оценивания и пороговые значения

Процент выполнения	Категория	Рекомендуемое кадровое решение
90% – 100%	«Отлично» / «Высокий уровень»	Включение в кадровый резерв, повышение категории
75% – 89%	«Хорошо» / «Базовый уровень»	Аттестован, соответствует должности
50% – 74%	«Удовлетворительно» / «Средний уровень»	Назначение повторной аттестации, план развития
0% – 49%	«Неудовлетворительно» / «Низкий уровень»	Направление на обучение, дисциплинарные меры

В приведшей главе были выделены временные режимы тестирования. При реализации методики оценки необходимо учитывать, что истечение времени должно трактоваться однозначно. (Таблица 9)

Таблица 9 – Учёт временных ограничений в оценке результатов

Сценарий	Действие системы	Запись результата
Сотрудник завершил тест досрочно	Сохранить все ответы как есть	«Завершено успешно (досрочно)»
Истекло общее время теста	Автоматическая отправка текущих ответов	«Завершено по таймауту»
Истекло время на отдельный вопрос	Автоматический переход к следующему вопросу, текущий ответ не сохраняется	Вопрос засчитывается как 0 баллов
Сотрудник закрыл вкладку браузера	При повторном входе – восстановление сессии (если тест не завершён)	Не завершённые тесты сохраняются в статусе «в процессе»

Одна из ключевых задач аттестационного тестирования – предоставление сотруднику содержательной обратной связи, а не только числового балла [8]. Предлагается двухуровневая система обратной связи:

Уровень 1. Индивидуальный отчёт для сотрудника.

После завершения теста сотрудник должен видеть:

1. Общий процент выполнения и итоговую категорию (например, «Хорошо»).
2. Количество правильных ответов из общего числа (8 из 10).

3. Детализацию по компетенциям / разделам (если тест структурирован).

4. Время, затраченное на тест.

Уровень 2. Сводный отчёт для руководителя (HR).

Руководитель / HR-менеджер получает доступ к расширенному отчёту, включающему:

1. Список всех сотрудников с их результатами и категориями.

2. Статистику по каждому вопросу (какой процент сотрудников ответил верно – для выявления «провальных» тем).

3. Распределение результатов.

2. ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Проектирование ИС

2.1.1 Описание функционала информационной системы

Перед началом разработки информационной системы, предназначенной для автоматизации процесса входного тестирования сотрудников для предприятия ООО «Полимер», был проведён анализ предметной области [11]. Предприятие регулярно ищет новых сотрудников на различные профессии. Каждая из профессий предполагает свой набор профессиональных вопросов для проверки компетентности. Успешное подтверждение владения этими знаниями через процедуру тестирования является обязательным и неотъемлемым условием для последующего допуска сотрудника к самостоятельному выполнению трудовых функций.

Существовавший до начала разработки системы регламент проведения входного тестирования базировался исключительно на ручном труде и бумажном документообороте. В рамках этого процесса подготавливались и распечатывались контрольные бланки с тестовыми вопросами, которые затем заполнялись новым сотрудником от руки. После завершения заполнения полученные результаты проверялись ответственным лицом вручную, и лишь затем итоговые оценки и сопутствующая информация вносились в общий журнал учёта результатов тестирования. Применявшийся подход характеризовался целым рядом системных недостатков, среди которых особенно выделялись значительные временные затраты на всех этапах – от печати бланков до подсчёта баллов, высокий риск случайных ошибок при ручной проверке ответов, а также существенные трудности, возникавшие при попытке

сформировать сводную аналитику или статистическую отчётность по результатам тестирования за произвольный период. Указанные обстоятельства объективно обосновывали необходимость перехода к автоматизированному решению.

В ходе исследования предметной области были созданы следующие пользовательские истории (User Stories) [11, 16]:

- как сотрудник, я хочу подготовиться к вопросам до начала теста;
- как сотрудник, я хочу проходить тест по выбранной профессии;
- как сотрудник, я хочу знать свой результат тестирования.

На основе пользовательских историй были созданы функциональные требования [11, 16]:

- сотрудник должен иметь возможность подготовки к тесту до его начала.
- сотрудник должен видеть весь список тестов;
- сотрудник должен иметь возможность проходить тест;
- при прохождении теста система должна полностью отображать вопросы на одной странице;
- на странице вопроса должны отображаться: текст вопроса и варианты ответа;
- система должна сохранять имена сотрудников в зашифрованном виде;
- по завершении теста система должна отображать результат: процент выполнения, итоговую категорию («Сдал», «Не сдал»), количество правильных ответов.

Нефункциональные требования [11, 16]:

- система должна сохранять работоспособность при одновременной работе с несколькими активными пользователями;

- время генерации страницы с вопросом теста не должно превышать 1 секунды при штатной нагрузке;
- время обработки и сохранения теста не должно превышать 1 секунды;
- система должна быть защищена от SQL-инъекций. Все запросы к PostgreSQL выполняются только через параметризованные PDO-запросы;
- система должна адаптироваться к разным размерам экрана;
- система должна быть легко расширяемой;

2.1.2 Концептуальное проектирование информационной системы

На рисунке 5 представлена диаграмма «сущность-связь», отображающую концептуальную модель базы данных для сайта тестирования сотрудников [23]. Она изображает основные сущности (таблицы) и связи между ними.

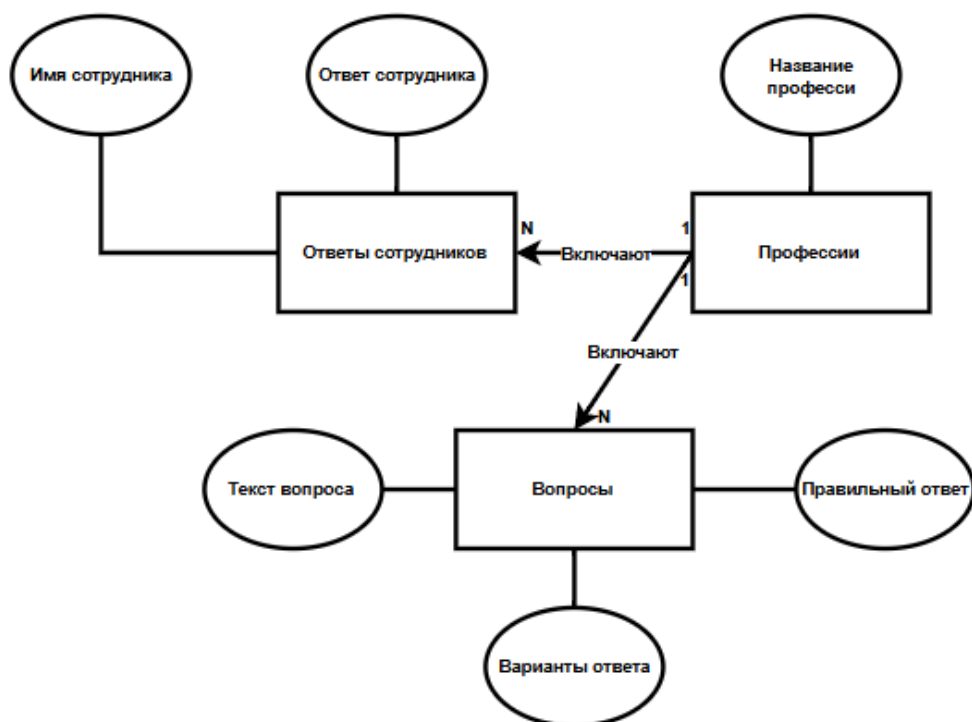


Рисунок 5. Диаграмма сущность-связь

Сущности:

- профессии: название профессии;
- вопросы: профессия, текст вопроса, варианты ответа, правильный ответ;
- ответы сотрудников: профессия, имя сотрудника, ответ сотрудника.

2.1.3 Проектирование структуры базы данных

Для проектирования базы данных сайта тестирования сотрудников будет использоваться реляционная база данных и была разработана ER-диаграмма, (Рисунок 6) которая отображает сущности системы и отношения между ними[5].

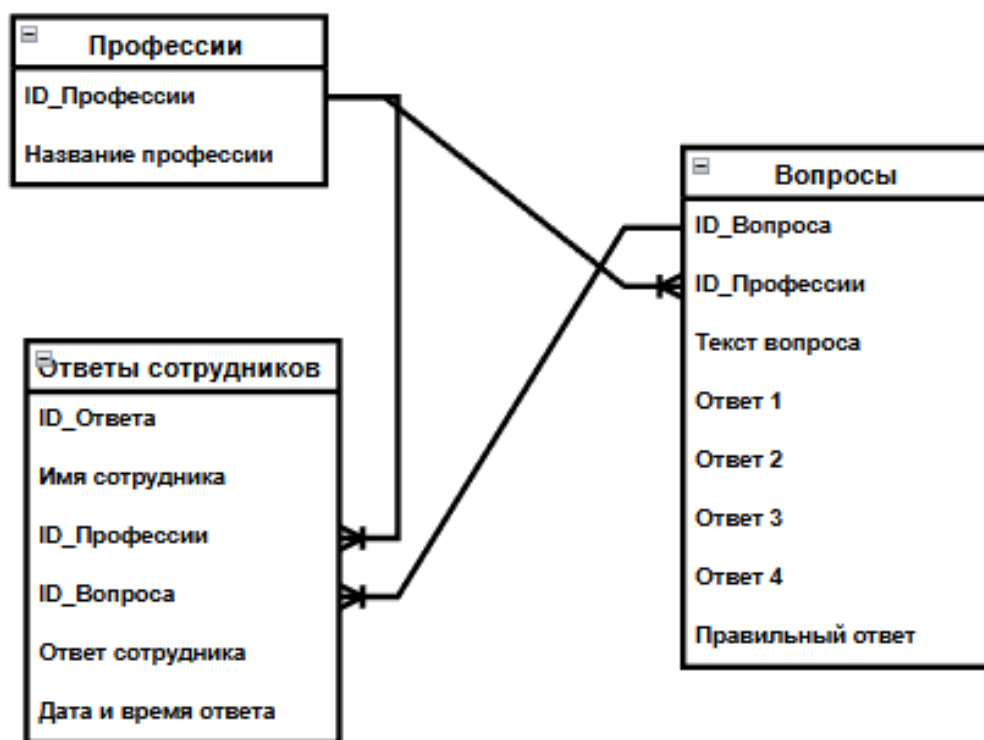


Рисунок 6. ER-Диаграмма базы данных

Отношения 1: N (один ко многим):

- профессии – Вопросы: у одной профессии, может быть несколько вопросов;
- вопросы – Ответы сотрудников: на один вопрос, может быть дано несколько ответов;
- профессии – Ответы сотрудников: у одной профессии, может быть дано несколько ответов.

2.1.4 Разработка структуры с учетом модели данных

Разработка логической структуры базы данных как ключевой этап проектирования информационной системы Проектирование логической структуры базы данных (БД) является одним из наиболее ответственных этапов создания информационной системы, поскольку именно здесь формируются базовые принципы организации данных, определяются семантические связи между объектами предметной области и фиксируются ограничения, обеспечивающие целостность, непротиворечивость и согласованность информации на протяжении всего жизненного цикла системы [3, 23].

Обоснование выбора реляционной модели данных: для разрабатываемого решения выбрана реляционная модель данных. Данный выбор обусловлен её наилучшим соответствием предъявляемым требованиям: строгая структурированность, развитые механизмы контроля целостности, гибкость при формировании запросов и поддержка стандартных средств взаимодействия [3].

Основные достоинства реляционного подхода в контексте проекта:

- чёткая организация данных: информация представляется в виде плоских таблиц, состоящих из строк и столбцов с фиксированными типами данных. Такая формализация упрощает понимание логической

схемы как разработчиками, так и будущими администраторами системы, а также облегчает интеграцию с прикладным программным обеспечением;

- поддержка целостности на уровне схемы Использование первичных ключей (PRIMARY KEY) для уникальной идентификации каждой записи, внешних ключей (FOREIGN KEY) для реализации ссылочной целостности, а также ограничений (CHECK, NOT NULL, UNIQUE) позволяет предотвратить появление противоречивых, дублирующихся или некорректных записей ещё на этапе ввода данных, не перекладывая эту логику на прикладной код;

- гибкость запросов и независимость от физического хранения Мощный и стандартизированный язык SQL обеспечивает выполнение выборки любой сложности: от простых фильтров до многотабличных соединений (JOIN), подзапросов, оконных функций и агрегирования [18, 19]. Отчёты и аналитические выборки могут быть построены без изменения физической структуры таблиц, что критически важно для систем с изменяющимися требованиями к аналитике.

Выбор системы управления базами данных: в качестве системы управления базами данных выбрана PostgreSQL. Выбор обоснован следующими преимуществами, имеющими критическое значение для корпоративного сайта для входного тестирования сотрудников:

- открытая лицензия и низкая совокупная стоимость владения PostgreSQL распространяется под свободной лицензией, что полностью исключает лицензионные отчисления. Это существенно снижает стоимость разработки, тестирования, промышленной эксплуатации и возможного масштабирования, особенно в корпоративной среде, где количество развёртываний может быть значительным [19];

- высокая надёжность и соответствие стандартам SQL. СУБД зарекомендовала себя как стабильное решение, устойчивое к высоким нагрузкам и сбоям. Поддерживаются полноценные ACID-транзакции

(атомарность, согласованность, изоляция, долговечность), что гарантирует корректность конкурентного доступа к данным тестирований [19];

– средства масштабирования – встроенные механизмы репликации позволяют создавать отказоустойчивые кластеры, а партиционирование крупных таблиц способствует сохранению производительности при росте данных [19].

Приведение базы данных к третьей нормальной форме: с целью устранения избыточности, предотвращения аномалий модификации и обеспечения целостности на уровне логической модели. База данных спроектирована в соответствии с принципами нормализации. Выполнено последовательное приведение к третьей нормальной форме (3NF).

Реализованные требования 3NF в рамках проекта [5, 23]:

1. Атомарность атрибутов (1НФ). Все атрибуты во всех таблицах являются скалярными и неделимыми в рамках используемой модели. Повторяющиеся группы отсутствуют. Составные значения вынесены в отдельные таблицы или представлены как отдельные столбцы с примитивными типами (INTEGER, VARCHAR, TIMESTAMP и т. д.).

2. Наличие первичного ключа (1НФ). Для каждой таблицы определён первичный ключ – уникальный идентификатор, однозначно идентифицирующий каждую запись. В качестве первичных ключей используются суррогатные автоинкрементные целочисленные идентификаторы (SERIAL/BIGSERIAL).

3. Устранение частичных зависимостей (2НФ). Для таблиц с составными первичными ключами проверено, что все неключевые атрибуты функционально полно зависят от всего ключа, а не от его части. Если выявлялась частичная зависимость, соответствующая группа атрибутов выносилась в отдельную таблицу. Это исключило аномалии, связанные с дублированием информации о части составного ключа.

4. Устранение транзитивных зависимостей (ЗНФ). Ни один неключевой атрибут не зависит от другого неключевого атрибута. Все неключевые атрибуты зависят исключительно от первичного ключа таблицы. Если обнаруживалась зависимость «неключевой атрибут → другой неключевой атрибут», это приводило к вынесению таких атрибутов в отдельную сущность, а в исходной таблице оставался только внешний ключ на неё. Тем самым достигнута минимальная избыточность и устранена возможность несогласованных обновлений.

На основе анализа были созданы основные сущности:

1. Таблица: professions (Профессии) (Таблица 13): хранит названия профессий.

Таблица 13 – Структура таблицы professions

Атрибут	Тип данных	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор
name	VARCHAR (100)	NOT NULL	Название профессии

2. Таблица: questions (Вопросы) (Таблица 14): хранит текст, ответы на вопросы, а также правильные ответы.

Таблица 14 – Структура таблицы questions

Атрибут	Тип данных	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор
profession_id	INTEGER	NOT NULL, REFERENCES professions(id)	Уникальный идентификатор профессии
question_text	VARCHAR (255)	NOT NULL	Тест вопроса
option1	VARCHAR (255)	NOT NULL	1 Вариант ответа
option2	VARCHAR (255)	NOT NULL	2 Вариант ответа
option3	VARCHAR (255)	NOT NULL	3 Вариант ответа
option4	VARCHAR (255)	NOT NULL	4 Вариант ответа
correct_option	INTEGER	NOT NULL	Правильный вариант ответа

3. Таблица: results (Вопросы) (Таблица 15): хранит ответы сотрудников.

Таблица 15 – Структура таблицы results

Атрибут	Тип данных	Ограничения	Описание
id	SERIAL	PRIMARY KEY	Уникальный идентификатор
name	VARCHAR (255)	NOT NULL	Имя сотрудника
profession_id	INTEGER	NOT NULL, REFERENCES professions(id)	Уникальный идентификатор профессии
question_id	INTEGER	NOT NULL REFERENCES questions(id)	Уникальный идентификатор вопроса
user_answer	INTEGER	NOT NULL	Ответ сотрудника
answer_date	TIMESTAMP	DEFAULT CURRENT_TIMES TAMP	Дата и время отправки ответа

2.2 Реализация информационной системы

2.2.1 Реализация базы данных

Реализация базы данных – ключевой этап разработки ИС: в ней хранятся данные о пользователях, тестах, вопросах, ответах и результатах. От качества реализации зависят производительность, целостность и масштабируемость системы [7].

На этом этапе создаются таблицы по утверждённой логической структуре, задаются ограничения (первичные/внешние ключи, проверки, значения по умолчанию) и индексы для производительности [19]. Для совместимости и миграции используются англоязычные имена объектов [7].

После создания базы данных и таблиц получаем следующую ER–диаграмму на рисунке 7.

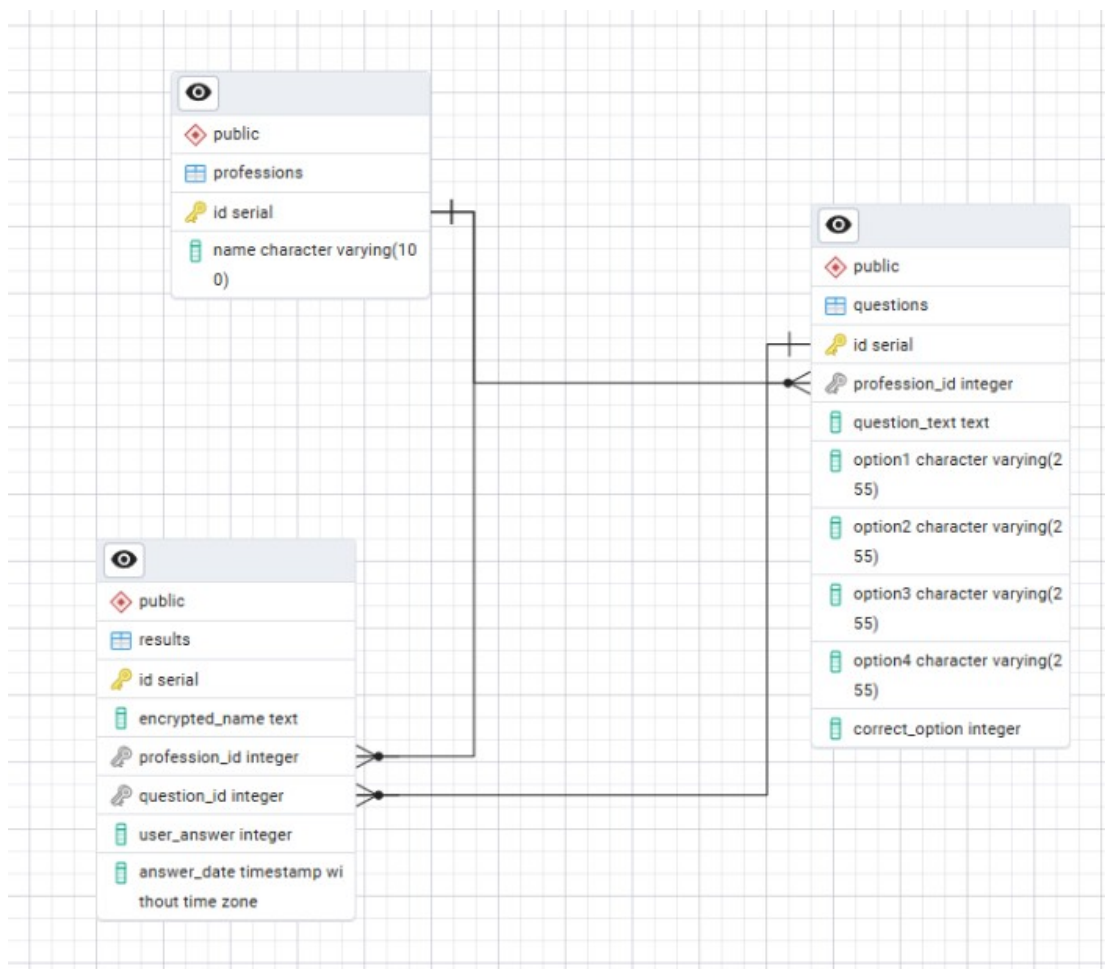


Рисунок 7. ER-диаграмма из СУБД Postgresql

На рисунке 8 представлен SQL-запрос для индексирования ID профессий в таблице вопросы, имя пользователя в таблице результаты, дата ответа в таблице результаты ID профессий в таблице результаты, ID вопроса в таблице результаты.

```

-- Индексы для оптимизации запросов
CREATE INDEX idx_question_professio ON questions(profession_id);
CREATE INDEX idx_results_encrypted ON results(encrypted_name);
CREATE INDEX idx_results_date ON results(answer_date);
CREATE INDEX idx_results_profession ON results(profession_id);
CREATE INDEX idx_results_question ON results(question_id);

```

Рисунок 8. Создание индексов

На рисунке 9 представлен SQL-запрос для включения расширения pgcrypto.

```
-- Включаем расширение pgcrypto
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

Рисунок 9. Включение расширения pgcrypto

На рисунке 10 представлен SQL-запрос для создания функции encrypt_name. Функция принимает текстовый параметр user_name и возвращает текстовое значение. В теле функции происходит шифрование входной строки. Для шифрования применяется фиксированный пароль 'polymer', предоставленный функции pgp_sym_encrypt. Результатом шифрования являются бинарные данные типа BYTEA, которые сохраняются в переменную encrypted_bytes. Затем функция преобразует эти бинарные данные в текстовую строку в кодировке Base64 с помощью функции encode. Итоговое значение возвращается вызывающему коду как обычный текст. Функция написана на языке PL/pgSQL и предназначена для безопасного хранения зашифрованных текстовых данных в текстовых полях таблиц.

```
-- Функция для шифрования имени
CREATE OR REPLACE FUNCTION encrypt_name(user_name TEXT)
RETURNS TEXT AS $$
DECLARE
    encrypted_bytes BYTEA;
BEGIN
    SELECT pgp_sym_encrypt(user_name, 'polymer') INTO encrypted_bytes;
    RETURN encode(encrypted_bytes, 'base64');
END;
$$ LANGUAGE plpgsql;
```

Рисунок 10. Создание функции encrypt_name

На рисунке 11 представлен SQL-запрос для создания функции `decrypt_name`. Функция принимает один входной параметр `encrypted_text` типа `TEXT`, который представляет собой зашифрованную строку в кодировке `Base64`. В теле функции сначала происходит декодирование из `Base64` в бинарный формат `BYTEA`. Затем полученные бинарные данные расшифровываются симметричным алгоритмом через функцию `pgp_sym_decrypt` с использованием пароля `polymer`. Результат возвращается как обычный текст. В случае возникновения любой ошибки на любом этапе обработки срабатывает блок исключений, который перехватывает исключение и возвращает строку «Ошибка дешифровки». Функция реализована на языке `PL/pgSQL`.

```
-- Функция для расшифровки имени
CREATE OR REPLACE FUNCTION decrypt_name(encrypted_text TEXT)
RETURNS TEXT AS $$
DECLARE
    encrypted_bytes BYTEA;
BEGIN
    SELECT decode(encrypted_text, 'base64') INTO encrypted_bytes;
    RETURN pgp_sym_decrypt(encrypted_bytes, 'polymer');
EXCEPTION
    WHEN others THEN
        RETURN 'Ошибка дешифровки';
END;
$$ LANGUAGE plpgsql;
```

Рисунок 11. Создание функции `decrypt_name`

2.2.2 Создание информационной системы

Создание информационной системы для входного тестирования сотрудников требует обоснованного выбора набора технологий, которые в совокупности образуют надёжную, безопасную и удобную в сопровождении платформу [16].

Выбор серверного языка программирования – PHP. Для реализации серверной логики был выбран язык PHP версии 8.4. Этот выбор обусловлен несколькими весомыми причинами. PHP является доминирующим языком в сегменте корпоративных веб-приложений средней размерности. Это означает, что для языка существует огромное сообщество разработчиков, тысячи готовых решений для типовых задач и, что особенно важно для данного проекта, исчерпывающая документация, переведённая на русский язык [10, 18].

Кроме того, PHP не требует компиляции и работает по модели «запрос – ответ» в рамках одного процесса[10]. Каждый HTTP-запрос от браузера сотрудника обрабатывается независимо: PHP-интерпретатор загружается, выполняет скрипт, генерирует HTML-страницу и завершает работу. Такая модель, с одной стороны, исключает проблемы с утечкой памяти, характерные для долгоживущих, а с другой – делает код предсказуемым и простым в отладке. Не менее важным аргументом в пользу PHP стала его встроенная поддержка работы с сессиями. И его реализация «из коробки» без подключения дополнительных библиотек существенно ускоряет разработку.

HTML выступает в качестве языка разметки для всех страниц системы [20]. Генерация HTML-кода выполняется непосредственно PHP-скриптами с использованием PHP-кода для вывода динамического содержимого – текста вопросов, вариантов ответов, результатов тестирования.

Для оформления пользовательского интерфейса используется CSS3. Каскадные таблицы стилей обеспечивают единообразное визуальное представление всех страниц системы – от формы входа до страницы с результатами тестирования. В рамках проекта разработана адаптивная вёрстка, позволяющая корректно отображать страницы на экранах

различного разрешения, включая мониторы с соотношением сторон 4:3, которые до сих пор могут встречаться на некоторых рабочих местах [20].

Для взаимодействия PHP с PostgreSQL используется расширение PDO_PGSQL, входящее в стандартную поставку PHP. PDO (PHP Data Objects) предоставляет единый интерфейс для работы с различными СУБД и, что самое важное, поддерживает параметризованные запросы, полностью исключая возможность SQL-инъекций [10,18]. В процессе разработки все запросы к базе данных формулируются с использованием подготовленных выражений, что гарантирует безопасность при работе с пользовательским вводом.

Пакетная структура веб-приложения представлена на рисунке 12.

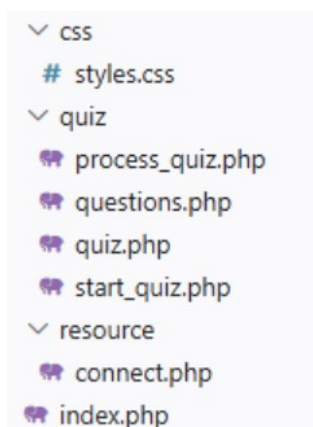


Рисунок 12. Пакетная структура приложения

Файл `connect.php` (ПРИЛОЖЕНИЕ 1) выполняет функцию установления соединения между PHP-скриптами и базой данных PostgreSQL, а также управляет сессиями пользователей.

В начале файла вызывается `session_start()`, что позволяет использовать механизм сессий для хранения временных данных, например информации о прохождении теста пользователем. Далее определяются параметры подключения к серверу баз данных. Эти параметры используются для создания объекта PDO (PHP Data Objects), который

обеспечивает безопасное и универсальное взаимодействие с базой данных[10].

Конструкция `try catch` обрабатывает возможные ошибки подключения [14]. В случае успешного соединения устанавливается режим сообщения об ошибках `ERRMODE_EXCEPTION`, который заставляет PDO генерировать исключения при любых сбоях выполнения запросов. После этого выполняется SQL-команда `SET client_encoding TO 'UTF8'`, гарантирующая корректную обработку кириллических символов и других Unicode-символов при обмене данными между приложением и базой данных. Если подключение не удалось, выполнение скрипта прерывается с выводом сообщения об ошибке через функцию `die()`. Полученный объект PDO сохраняется в переменной `$pdo`, которая затем используется во всех остальных скриптах для выполнения запросов к базе данных.

Файл `index.php` (ПРИЛОЖЕНИЕ 2) представляет собой главную страницу веб-приложения, (Рисунок 13) предназначенную для навигации пользователей по основным разделам системы тестирования.

При загрузке страницы пользователь видит контейнер с заголовком «Добро пожаловать» и три интерактивные кнопки, каждая из которых реализована в виде отдельной формы с методом GET. Первая кнопка «Вход для администратора» при нажатии перенаправляет пользователя на страницу `admin_panel/login_admin.php`, где происходит аутентификация административного персонала. Вторая кнопка «Пройти тест» ведёт на страницу `quiz/start_quiz.php`, где пользователь может выбрать профессию и начать тестирование. Третья кнопка «Посмотреть вопросы для подготовки» направляет на страницу `quiz/questions.php`, предоставляющую доступ к банку вопросов для самостоятельной подготовки [12,22].

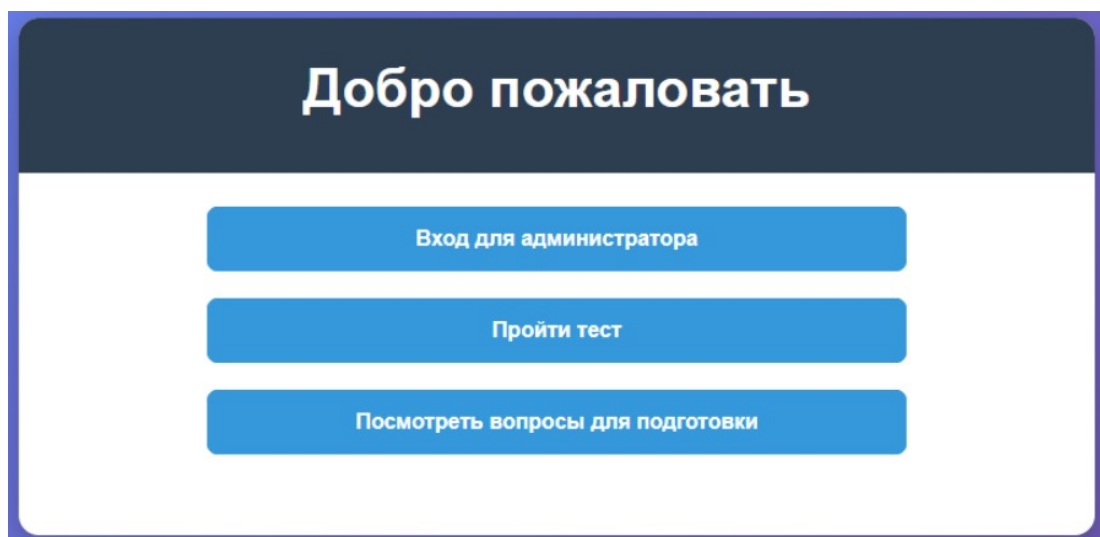


Рисунок 13. Интерфейс главной страницы

Файл `questions.php` (ПРИЛОЖЕНИЕ 3) реализует функциональность просмотра всех вопросов системы для подготовки к тестированию. (Рисунок 14) Данный скрипт не требует авторизации и доступен любому пользователю, перешедшему по соответствующей ссылке с главной страницы.

При загрузке страницы происходит подключение к базе данных через файл `connect.php` и выполняется запрос на получение полного списка профессий из таблицы `professions`. Полученные данные используются для построения интерфейса фильтрации. Скрипт также принимает GET-параметр `profession_id`, который определяет, для какой специальности будут показаны вопросы. Если параметр отсутствует или равен нулю, страница отображает только форму выбора профессии без списка вопросов.

При наличии корректного идентификатора профессии выполняется подготовленный запрос к базе данных. Он извлекает все вопросы из таблицы `questions` для указанной профессии, а также подгружает название профессии из связанной таблицы. Результаты сортируются по идентификатору вопроса. После получения данных происходит проверка:

если массив вопросов не пуст, на странице выводится заголовок с названием профессии и общим количеством вопросов, после чего каждый вопрос отображается в отдельном блоке с порядковым номером. Если вопросы не найдены, пользователю показывается соответствующее сообщение об отсутствии вопросов для выбранной специальности [12,22].

Вопросы для подготовки
Выберите профессию для просмотра вопросов

Выберите профессию:

Все профессии Программист Дизайнер

Маркетолог Менеджер

Вернуться

Вопросы для профессии: Программист
Количество вопросов: 6

Вопрос 1 - Какой язык используется для стилизации веб-страниц?

Вопрос 2 - Что означает SQL?

Вопрос 3 - Какой тег используется для создания ссылки в HTML?

Рисунок 14. Интерфейс просмотра всех вопросов

Файл `start_quiz.php` (ПРИЛОЖЕНИЕ 4) является точкой входа в процесс тестирования (Рисунок 15) и выполняет две основные функции: сбор первичных данных о пользователе и инициализацию сессии тестирования.

При загрузке страницы происходит подключение к базе данных через файл `connect.php` и выполняется запрос на получение всех записей из

таблицы professions. Полученный список специальностей используется для формирования выпадающего списка, из которого пользователь впоследствии выберет направление для проверки своих знаний.

Интерфейс страницы представляет собой веб-форму, содержащую два обязательных поля. Поле ввода текста предназначено для указания имени пользователя, которое в дальнейшем будет зашифровано и сохранено в базе данных вместе с результатами тестирования. Выпадающий список позволяет выбрать одну из профессий, предварительно загруженных из базы данных. Оба поля являются обязательными для заполнения, что обеспечивается атрибутом required в HTML-разметке.

После заполнения формы и нажатия кнопки «Начать тест» срабатывает JavaScript-обработчик события отправки формы. Он предотвращает стандартное поведение браузера по перезагрузке страницы и выполняет валидацию введенных данных. Если оба поля заполнены, скрипт программно перенаправляет пользователя на страницу quiz.php, передавая имя и идентификатор выбранной профессии в качестве GET-параметров. Передача данных осуществляется с использованием функции encodeURIComponent для корректной обработки специальных символов в имени пользователя.

Страница не сохраняет никаких данных в сессию или базу данных на этом этапе, выполняя исключительно функцию сбора информации. Вся последующая обработка, включая выборку случайных вопросов и шифрование персональных данных, происходит уже в следующих скриптах тестирования. Такой подход позволяет отделить процесс идентификации пользователя от непосредственно процедуры тестирования, упрощая архитектуру и поддержку кода [12,22].

Тестирование по профессиям

Выберите профессию и пройдите тест

Ваше имя:

Выберите профессию:

-- Выберите профессию --

Начать тест

Рисунок 15. Интерфейс начала тестирования

Файл `quiz.php` (ПРИЛОЖЕНИЕ 5) является основным компонентом процесса тестирования и отвечает за отображение вопросов и сбор ответов пользователя. (Рисунок 16)

При загрузке страница сначала проверяет наличие в GET-запросе двух обязательных параметров: имени пользователя (`name`) и идентификатора выбранной профессии (`profession`). Если хотя бы один из параметров отсутствует, скрипт немедленно выполняет перенаправление на стартовую страницу `start_quiz.php`, предотвращая некорректный доступ к тестированию.

После успешной валидации входных данных выполняется запрос к базе данных. Из таблицы `questions` случайным образом выбираются десять записей, соответствующих переданному идентификатору профессии. Использование функции `RANDOM()` в сочетании с `LIMIT 10` обеспечивает вариативность теста при каждом прохождении, что исключает заучивание порядка вопросов. Если для выбранной специальности не найдено ни

одного вопроса, выполнение скрипта прерывается с сообщением об ошибке.

Сформированный набор вопросов вместе с именем пользователя и идентификатором профессии сохраняется в сессионную переменную `$_SESSION['quiz_data']`. В этот же массив добавляется временная метка начала тестирования. Такой подход позволяет передать данные на страницу обработки результатов без использования скрытых полей формы или GET-параметров, что повышает безопасность и удобство реализации.

HTML-разметка генерирует форму, в которой последовательно выводятся все десять вопросов. Для каждого вопроса отображается его текст и четыре варианта ответа. Каждый вариант ответа представлен отдельным блоком `div` с классом `option`. К каждому вопросу прикреплено скрытое поле ввода, идентификатор которого формируется на основе идентификатора вопроса в базе данных. Это скрытое поле предназначено для хранения номера варианта, выбранного пользователем.

JavaScript-функция `selectOption` обрабатывает клики по блокам с вариантами ответов. При выборе опции функция сначала удаляет класс `selected` со всех вариантов текущего вопроса, обеспечивая визуальное снятие выделения с ранее выбранного ответа. Затем выбранному блоку присваивается класс `selected`, и соответствующее скрытое поле получает значение номера варианта от одного до четырёх. Данный механизм обеспечивает интуитивно понятный интерфейс без использования стандартных радиокнопок, что позволяет более гибко стилизовать элементы управления.

В нижней части формы размещена кнопка отправки данных. При её нажатии форма передаёт массив `answers` методом POST на скрипт `process_quiz.php`, где происходит вычисление результатов и сохранение ответов в базе данных. Завершение теста возможно только после того, как пользователь ответил на все вопросы, так как скрипт не реализует

проверку полноты заполнения формы на стороне клиента перед отправкой [12,22].

Тестирование
Пользователь: Никита

Вопрос 1: Какой язык используется для стилизации веб-страниц?

JavaScript

Python

CSS

HTML

Вопрос 2: Что такое переменная в программировании?

Константное значение

Именованная область памяти

Функция

Цикл

Рисунок 16. Интерфейс прохождения теста

Файл `process_quiz.php` (ПРИЛОЖЕНИЕ 6) является обработчиком результатов тестирования. (Рисунок 17) Он принимает данные, переданные из формы `quiz.php`, производит вычисление правильных ответов, сохраняет результаты в базе данных с применением шифрования и отображает итоговую статистику пользователю.

При выполнении скрипта первым шагом происходит проверка наличия в сессии массива `quiz_data`, а также массива `POST['answers']`. Если какое-либо из этих условий не выполняется, это означает, что пользователь попытался обратиться к обработчику напрямую или сессия была утеряна. В такой ситуации выполнение скрипта прерывается, и

пользователь перенаправляется обратно на страницу `start_quiz.php`, что предотвращает некорректную обработку.

После успешной валидации данные из сессии извлекаются в локальные переменные, включая имя пользователя, идентификатор профессии и массив вопросов, которые были заданы во время тестирования. Из массива `$_POST['answers']` извлекаются ответы пользователя, где ключами выступают идентификаторы вопросов, а значениями – номера выбранных вариантов от одного до четырёх.

Следующим этапом выполняется шифрование имени пользователя. Для этого используется подготовленный запрос к функции `encrypt_name`, созданной в базе данных PostgreSQL. Запрос принимает переданное имя и возвращает зашифрованную строку в формате Base64. Данный подход гарантирует, что персональные данные пользователей хранятся в защищённом виде в таблице `results`.

Далее скрипт инициирует цикл по всем вопросам, сохранённым в сессии. Для каждого вопроса производится сравнение ответа пользователя с правильным вариантом, хранящимся в поле `correct_option`. Параллельно с этим выполняется операция вставки записи в таблицу `results`. В базу данных заносятся зашифрованное имя пользователя, идентификатор профессии, идентификатор вопроса и выбранный пользователем ответ. Каждая запись автоматически получает временную метку благодаря значению по умолчанию `CURRENT_TIMESTAMP`, заданному в структуре таблицы. По завершении цикла подсчитывается общее количество правильных ответов.

На основе числа правильных ответов вычисляется итоговый процент. Производится округление до целого числа, и результат сохраняется в переменной `$score`. Сессионная переменная `quiz_data` уничтожается с помощью `unset`, чтобы предотвратить повторную отправку тех же данных при возможном обновлении страницы.

Завершающая часть скрипта генерирует HTML-страницу с результатами тестирования. На ней отображается имя пользователя, набранный процент и количество правильных ответов из общего числа вопросов. Условное оформление использует цветовую индикацию: зелёный цвет и текст «вы сдали» выводятся при достижении порога в 70 процентов и выше, в противном случае отображается красный цвет и текст «вы не сдали». Визуальная шкала прогресса заполняется на ширину, соответствующую полученному проценту. В нижней части страницы размещается кнопка для возвращения на главную страницу [12,22].

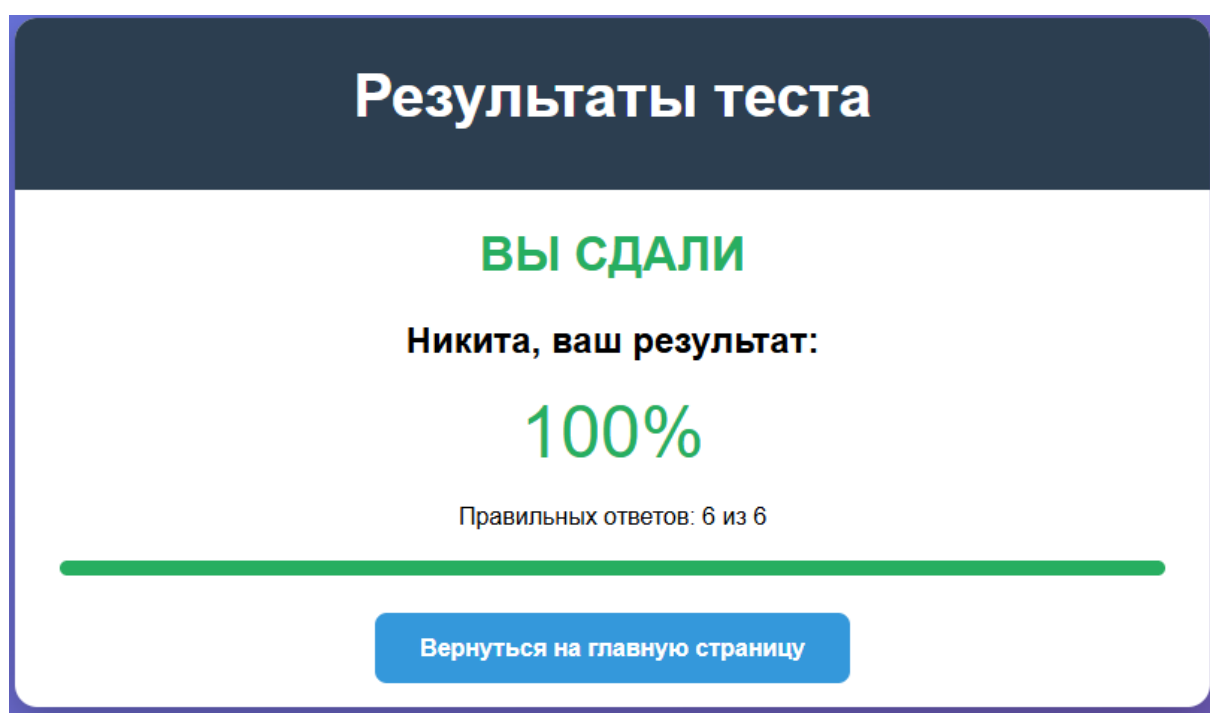


Рисунок 17. Интерфейс результата теста

2.3 UML-Диаграмма

В ходе создания системы разработана диаграмма последовательности. Диаграмма отражают порядок обмена сообщениями между сотрудником, фронтенд, бекенд и базой данных [11].

На рисунке 18 изображена диаграмма последовательности иллюстрирующая процесс прохождения тестирования.

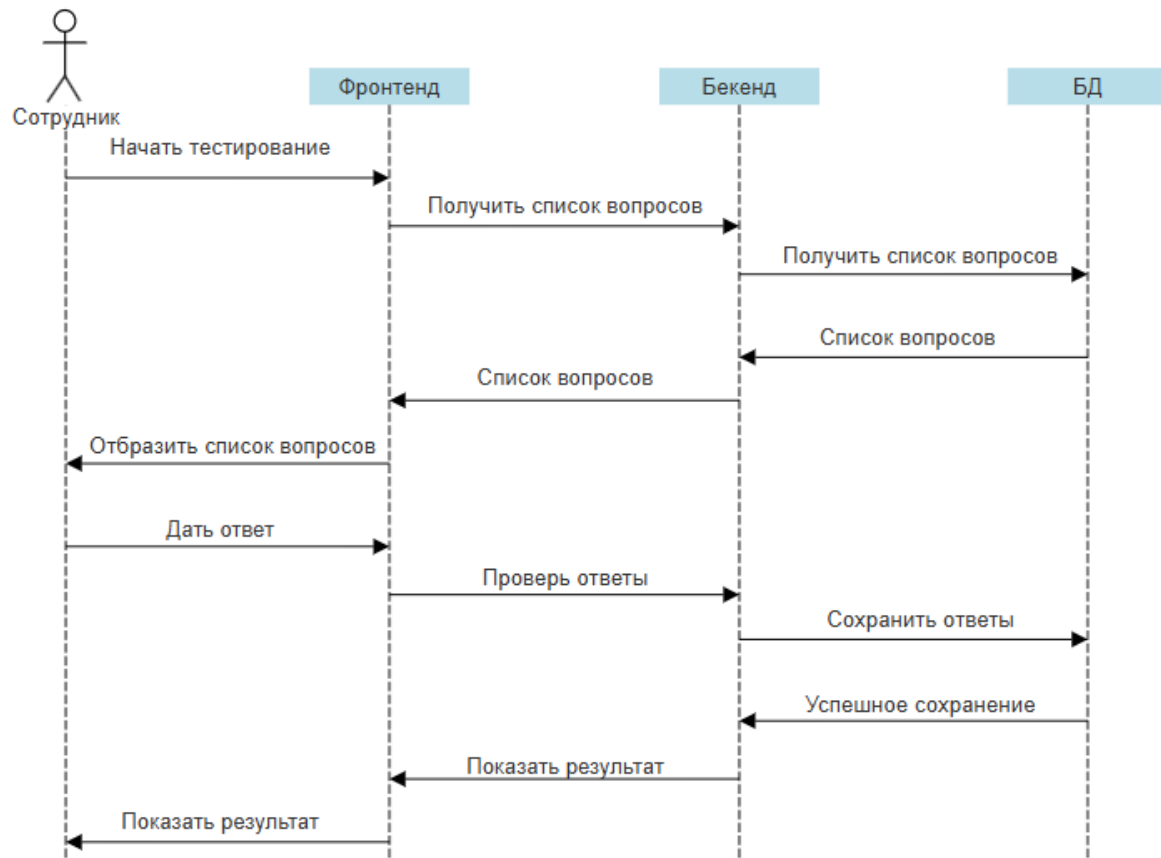


Рисунок 18. Диаграмма последовательности тестирования

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломной работы была достигнута поставленная цель: создан корпоративный веб-сайт для автоматизации процесса тестирования работников предприятия «Полимер».

В рамках исследования была подробно изучена и описана предметная область – процесс входного тестирования сотрудников.

На основе выявленных потребностей были сформулированы функциональные и нефункциональные требования к информационной системе. В соответствии с ними разработан проект информационной системы и структуры базы данных, которая позволяет эффективно хранить вопросы, варианты ответов, результаты пользователей.

В результате практической разработки создан действующий корпоративный веб-сайт, который обеспечивает полную автоматизацию тестирования и проверки тестов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Российская Федерация. Законы. О персональных данных: Федеральный закон № 152-ФЗ: принят Государственной Думой 8 июля 2006 года: одобрен Советом Федерации 14 июля 2006 года. – Москва: Кремль, 2006. – 26 с.
2. Аванесов, В. С. Композиция тестовых заданий / В. С. Аванесов. – 4-е изд., перераб. и доп. – Москва : Центр тестирования, 2018. – 300 с.
3. Агафонов, А. А. Основы технологий баз данных : учебное пособие / А. А. Агафонов, А. М. Белов. – Самара: Издательство Самарского университета, 2023. – 304 с.
4. Базаров, Т. Ю. Управление персоналом : учебник для вузов / Т. Ю. Базаров. – 2-е изд., стер. – Москва : Академия, 2020. – 357 с.
5. Базы данных: проектирование, реализация и сопровождение. Теория и практика / Т. Коннолли, К. Бегг. – М. : Вильямс, 2020. – 624 с.
6. Волков, А. А. React и PHP: разработка современных вебприложений / А. А. Волков. – М. : ДМК-Пресс, 2023. – 264 с.
7. Ганди, С. PostgreSQL для профессионалов / С. Ганди, Б. Момджи. – М. : Лори, 2021. – 420 с.
8. Гусев, А. Н. Психологические тесты: от простого к сложному / А. Н. Гусев. – Москва : Когито-Центр, 2020. – 312 с.
9. Дронов, В. А. PHP 7, MySQL и Dreamweaver: разработка интерактивных Web-сайтов : учебное пособие / В. А. Дронов. – 2-е изд., перераб. и доп. – Санкт-Петербург: БХВ-Петербург, 2022. – 544 с.
10. Дэвис, М. Изучаем PHP 8 / М. Дэвис. – Санкт-Петербург : Питер, 2022. – 512 с.
11. Козлов, В. С. Управление IT-проектами: от планирования до реализации / В. С. Козлов. – М. : Юрайт, 2022. – 245 с.

12. Коляда, М. Г. Основы веб-разработки на PHP / М. Г. Коляда. – Минск : БГУ, 2019. – 204 с.
13. Конструктор онлайн-опросов и тестов Testograf [Электронный ресурс]. – URL: <https://www.testograf.ru/> (дата обращения 20.04.2026)
14. Леннинг, Д. Д. PHP и PostgreSQL. Разработка веб-приложений / Д. Д. Леннинг, А. Б. Габасов. – М. : ДМК-Пресс, 2021. – 318 с.
15. Локхарт, Дж. Современный PHP: новые возможности и лучшие практики / Дж. Локхарт. – М. : Диалектика, 2022. – 352 с.
16. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. – Санкт-Петербург : Питер, 2021. – 352 с.
17. Матвеев, С. А. Технологии веб-разработки: полный цикл создания веб приложений / С. А. Матвеев. – СПб. : БХВ-Петербург, 2023. – 416 с.
18. Официальная документация PHP [Электронный ресурс]. – URL: <https://www.php.net/manual/ru/> (дата обращения: 14.04.2026).
19. Официальная документация PostgreSQL [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/> (дата обращения: 15.04.2026).
20. Семенова, Е. П. Проектирование пользовательских интерфейсов для бизнес-систем / Е. П. Семенова. – М. : Форум, 2022. – 208 с.
21. Смирнов, И. В. Проектирование веб-интерфейсов для бизнес-систем / И. В. Смирнов. – М. : Интернет-университет информационных технологий, 2023. – 268 с.
22. Тепляков, К. В. Веб-разработка на PHP и PostgreSQL / К. В. Тепляков. – М. : Бином, 2022. – 284 с.
23. Токмаков, Г. П. Базы данных: Модели и структуры данных, язык SQL, программирование баз данных : учебное пособие / Г. П. Токмаков. – Ульяновск: УлГТУ, 2021. – 362 с.

24. Git: Официальная документация по контролю версий [Электронный ресурс]. – URL: <https://git-scm.com/doc> (дата обращения: 05.04.2026)

25. Google Формы [Электронный ресурс]. – URL: <https://www.google.com/forms/> (дата обращения 21.04.2026).

26. Habr.com: Базы данных и PostgreSQL [Электронный ресурс]. – URL: <https://habr.com/ru/hub/postgresql/> (дата обращения: 21.04.2026).

27. Habr.com: Веб-разработка на PHP [Электронный ресурс]. – URL: <https://habr.com/ru/hub/php/> (дата обращения: 21.04.2026).

28. iSpring – Конструктор электронных онлайн-курсов [Электронный ресурс]. – URL: <https://www.ispring.ru/> (дата обращения 21.04.2026).

29. Moodle – Open-source learning platform Moodle.org [Электронный ресурс]. – URL: <https://moodle.org> (дата обращения 20.04.2026).

30. W3Schools: SQL и PHP Tutorials [Электронный ресурс]. – URL: <https://www.w3schools.com/> (дата обращения: 20.04.2026).

```
<?php
session_start();
$host = "localhost";
$dbname = "polymer";
$username = "postgres";
$password = "postgres";

try {
    $pdo = new PDO("pgsql:host=$host;dbname=$dbname", $username, $password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $pdo->exec("SET client_encoding TO 'UTF8'");
} catch(PDOException $e) {
    die("Ошибка подключения: " . $e->getMessage());
}
?>
```

Код подключения к базе

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Добро пожаловать</title>
  <link rel="stylesheet" href="css/styles.css">
</head>
<body>

  <div class="container">
    <div class="header">
      <h1>Добро пожаловать</h1>
    </div>
    <div class="content">
      <div class="content-index">

        <div class="content-btn">
          <form action="admin_panel/login_admin.php" method="GET">
            <button type="submit" class="btn">Вход для администратора</button>
          </form></div>

          <div class="content-btn">
            <form action="quiz/start_quiz.php" method="GET">
              <button type="submit" class="btn">Пройти тест</button>
            </form></div>

          <div class="content-btn">
            <form action="quiz/questions.php" method="GET">
              <button type="submit" class="btn">Посмотреть вопросы для подготовки</button>
            </form></div>

          </div>
        </div>
      </div>
    </body>
  </html>

```

Код главной страницы

```

<?php
require_once '../resource/connect.php';

$stmt = $pdo->query("SELECT * FROM professions ORDER BY id");
$professions = $stmt->fetchAll(PDO::FETCH_ASSOC);

$professionId = isset($_GET['profession_id']) ? (int)$_GET['profession_id'] : 0;

$questions = [];
$professionName = '';

if ($professionId > 0) {
    $stmt = $pdo->prepare("
        SELECT q.*, p.name as profession_name
        FROM questions q
        LEFT JOIN professions p ON q.profession_id = p.id
        WHERE q.profession_id = ?
        ORDER BY q.id
    ");
    $stmt->execute([$professionId]);
    $questions = $stmt->fetchAll(PDO::FETCH_ASSOC);

    if (!empty($questions)) {
        $professionName = $questions[0]['profession_name'];
    }
}
?>

```

Код серверной части страницы вопросов для подготовки

```

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Вопросы для подготовки</title>
  <link rel="stylesheet" href="../../css/styles.css">
</head>
<body> <div class="container preparation-container"> <div class="header">
  <h1>Вопросы для подготовки</h1> <p>Выберите профессию для просмотра вопросов</p>
</div> <div class="content"> <div class="profession-selector">
  <h3>Выберите профессию:</h3>
  <div class="selector-grid">
    <div class="profession-btn <?=$professionId == 0 ? 'active' : '' ?>"
      onclick="window.location.href='questions.php'"
      >Все профессии </div>
    <?php foreach($professions as $prof): ?>
      <div class="profession-btn <?=$professionId == $prof['id'] ? 'active' : '' ?>"
        onclick="window.location.href='questions.php?profession_id=<?=$prof['id'] ?>'>"
        <?=$htmlspecialchars($prof['name']) ?> </div>
    <?php endforeach; ?> </div> </div>
  <div style="text-align: center; margin-bottom: 30px;">
    <a href="../../index.php" class="btn">Вернуться </a> </div>
  <?php if(!empty($questions)): ?>
    <div class="questions-header">
      <h2>Вопросы для профессии: <?=$htmlspecialchars($professionName) ?></h2>
      <p>Количество вопросов: <?=$count($questions) ?></p></div>
    <?php foreach($questions as $index => $question): ?>
      <div class="question-card">
        <div class="question-text"> Вопрос <?=$index + 1 ?> - <?=$nl2br(htmlspecialchars($question['question_text'])) ?> </div> </div>
      <?php endforeach; ?> <?php elseif($professionId > 0): ?>
        <div class="empty-state">
          <h3>Вопросы не найдены</h3>
          <p>Для выбранной профессии пока нет вопросов.</p>
          <p>Попробуйте выбрать другую профессию.</p>
        </div>
      <?php else: ?>
        <div class="empty-state">
          <h3>Выберите профессию</h3>
          <p>Для просмотра вопросов выберите профессию из списка выше.</p>
          <p>Вы увидите все доступные вопросы для подготовки.</p> </div> <?php endif; ?> </div></div>
</body>
</html>

```

Код клиентской части страницы вопросов для подготовки

```

<?php
require_once '../resource/connect.php';

// Получаем список профессий
$stmt = $pdo->query("SELECT * FROM professions");
$professions = $stmt->fetchAll(PDO::FETCH_ASSOC);
?>

<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Тестирование профессий</title>
    <link rel="stylesheet" href="../css/styles.css">
</head>
<body>
    <div class="container">
        <div class="header">
            <h1>Тестирование по профессиям</h1>
            <p>Выберите профессию и пройдите тест</p>
        </div>
        <div class="content">
            <form id="userForm">
                <div class="form-group">
                    <label for="userName">Ваше имя:</label>
                    <input type="text" id="userName" name="userName" required>
                </div>
                <div class="form-group">
                    <label for="profession">Выберите профессию:</label>
                    <select id="profession" name="profession" required>
                        <option value="">-- Выберите профессию --</option>
                        <?php foreach($professions as $profession): ?>
                            <option value="{ $profession['id'] }">{ $profession['name'] }</option>
                        <?php endforeach; ?>
                    </select>
                </div>
                <button type="submit" class="btn">Начать тест</button>
            </form>
        </div>
    </div>
    <script>
        document.getElementById('userForm').addEventListener('submit', function(e) {
            e.preventDefault();
            const userName = document.getElementById('userName').value;
            const profession = document.getElementById('profession').value;

            if(userName && profession) {
                window.location.href = '../quiz/quiz.php?name={encodeURIComponent(userName)}&profession={profession}';
            }
        });
    </script>
</body>
</html>

```

Код начала тестирования

```

<?php
require_once '../resource/connect.php';

if(!isset($_GET['name']) || !isset($_GET['profession'])) {
    header('Location: start_quiz.php');
    exit();
}

$userName = $_GET['name'];
$professionId = $_GET['profession'];

// Получаем 10 случайных вопросов для выбранной профессии
$stmt = $pdo->prepare("
    SELECT * FROM questions
    WHERE profession_id = ?
    ORDER BY RANDOM()
    LIMIT 10
");
$stmt->execute([$professionId]);
$questions = $stmt->fetchAll(PDO::FETCH_ASSOC);

if(empty($questions)) {
    die("Вопросы для этой профессии не найдены.");
}

// Сохраняем вопросы в сессии
$_SESSION['quiz_data'] = [
    'user_name' => $userName,
    'profession_id' => $professionId,
    'questions' => $questions,
    'start_time' => date('Y-m-d H:i:s')
];
?>

```

Код серверной части тестирования

```

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Тестирование</title>
  <link rel="stylesheet" href="../css/styles.css">
</head>
<body>
  <div class="container">
    <div class="header">
      <h1>Тестирование</h1>
      <p>Пользователь: <?= htmlspecialchars($userName) ?></p>
    </div>
    <div class="content">
      <form id="quizForm" action="process_quiz.php" method="POST">
        <?php foreach($questions as $index => $question): ?>
          <div class="question">
            <h3>Вопрос <?= $index + 1 ?>: <?= htmlspecialchars($question['question_text']) ?></h3>
            <div class="options">
              <?php for($i = 1; $i <= 4; $i++): ?>
                <div class="option" onclick="selectOption(this, <?= $question['id'] ?>, <?= $i ?>)">
                  <?= htmlspecialchars($question['option' . $i]) ?>
                </div>
              <?php endfor; ?>
            </div>
            <input type="hidden" name="answers[<?= $question['id'] ?>]" id="answer_<?= $question['id'] ?>">
          </div>
        <?php endforeach; ?>
        <button type="submit" class="btn btn-success">Завершить тест</button>
      </form>
    </div>
  </div>
  <script>
    function selectOption(element, questionId, optionNumber) {
      // Снимаем выделение с других вариантов в этом вопросе
      const options = element.parentElement.querySelectorAll('.option');
      options.forEach(opt => opt.classList.remove('selected'));
      // Выделяем выбранный вариант
      element.classList.add('selected');
      // Сохраняем выбор в скрытом поле
      document.getElementById('answer_' + questionId).value = optionNumber;
    }
  </script>
</body>
</html>

```

Код клиентской части тестирования

```

<?php
require_once '../resource/connect.php';

if(!isset($_SESSION['quiz_data']) || !isset($_POST['answers'])) {
    header('Location: start_quiz.php');
    exit();
}

$quizData = $_SESSION['quiz_data'];
$userAnswers = $_POST['answers'];
$username = $quizData['user_name'];
$professionId = $quizData['profession_id'];
$questions = $quizData['questions'];

$correctAnswers = 0;
$totalQuestions = count($questions);

// Альтернативный вариант - шифрование через PostgreSQL функцию
$stmt = $pdo->prepare("SELECT encrypt_name(?) as encrypted_name");
$stmt->execute([$username]);
$result = $stmt->fetch(PDO::FETCH_ASSOC);
$pgEncryptedName = $result['encrypted_name'];

// Сохраняем ответы и подсчитываем правильные
foreach($questions as $question) {
    $questionId = $question['id'];
    $userAnswer = isset($userAnswers[$questionId]) ? (int)$userAnswers[$questionId] : 0;
    $correctAnswer = $question['correct_option'];

    // Сохраняем ответ в базу данных с шифрованным именем
    $stmt = $pdo->prepare("
        INSERT INTO results (encrypted_name, profession_id, question_id, user_answer)
        VALUES (?, ?, ?, ?)
    ");
    $stmt->execute([$pgEncryptedName, $professionId, $questionId, $userAnswer]);

    if($userAnswer === $correctAnswer) {
        $correctAnswers++;
    }
}

$score = round(($correctAnswers / $totalQuestions) * 100);

// Очищаем сессию
unset($_SESSION['quiz_data']);
?>

```

Код серверной части результатов теста

```

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Результаты теста</title>
  <link rel="stylesheet" href="../css/styles.css">
</head>
<body>
  <div class="container">
    <div class="header">
      <h1>Результаты теста</h1>
    </div>

    <div>
      <div class="result">
        <?php if($score >= 70): ??>
          <div class="result-score"><h1 style="color: #27ae60; font-size: 32px;">ВЫ СДАЛИ</p></div>
        <?php else: ??>
          <div class="result-score"><h1 style="color: #e74c3c; font-size: 32px;">ВЫ НЕ СДАЛИ</p></div>
        <?php endif; ??>

        <h2><?= htmlspecialchars($userName) ?>, ваш результат:</h2>
        <div class="score"><?= $score ?>%</div>
        <p>Правильных ответов: <?= $correctAnswers ?> из <?= $totalQuestions ?></p>

        <div class="progress">
          <div class="progress-bar" style="width: <?= $score ?>%"></div>
        </div>

        <div style="padding-top: 20px;"><a href="../index.php" class="btn">Вернуться на главную страницу</a></div>
      </div>
    </div>
  </div>
</body>
</html>

```

Код клиентской части результатов теста